Robot Perception, Localization and Navigation

Sen Wang

Neoscholar - 2025

Robot

Motion Planning and Navigation

Outline

1. Motion Planning and Formulation

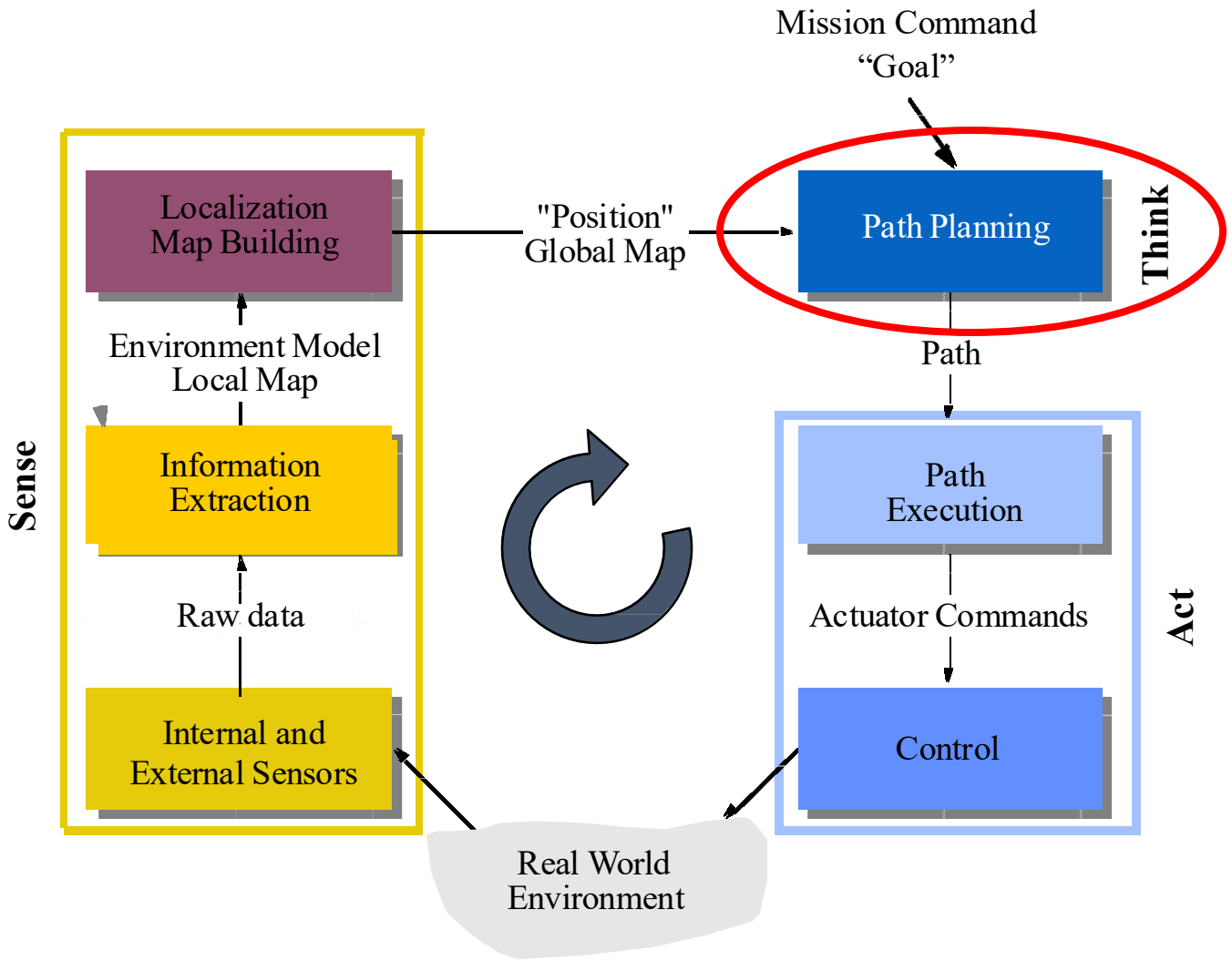
2. Global Path Planning

3. Local Path Planning (Obstacle Avoidance)

4. ROS Navigation Stack and Packages

Disclaimer: These slides can include material from different sources.
We would like to thank the colleagues who shared the resources used in these slides.

From Perception to Action for Full Autonomous Navigation



Planning

- ***Planning:*** Sequence of actions to achieve a given goal (usually without timing specifications), or towards achieving a given goal by achieving a set of sub-goals

- ***Scheduling:*** Planning + Timing (usually without space considerations)

- ***Task planning:*** Sequence of actions that accomplish a large goal (e.g., building a house)

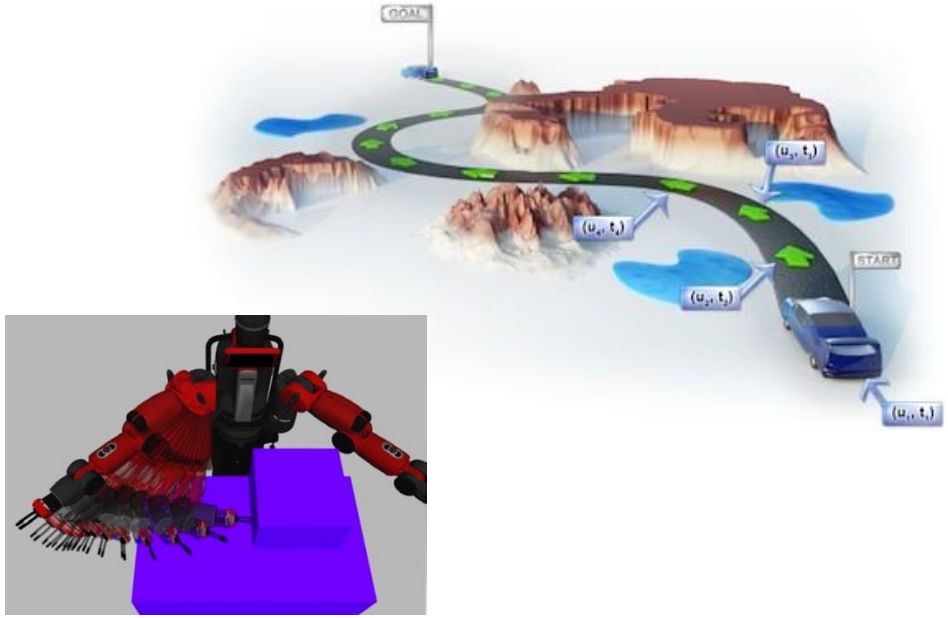
- ***Motion Planning (our focus):*** Generation of motions through space …

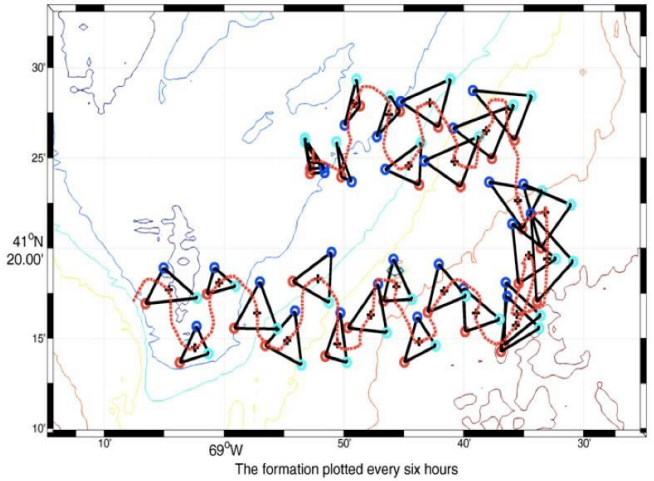
Planning is a **top-down** approach to problem solving that requires a (reliable) ***model*** of the world / problem, to be able to effectively *reason* about it

Motion Planning

Motion Planning: specification of motion through space (and time)

- **Path planning**: Generation of <u>a (feasible) path</u> for going from configuration A to B
- **Trajectory planning:** Generation of <u>a path</u> for going from A to B specifying <u>how to move based on velocity, time, differential constraints and dynamics</u>
- Coverage planning: Visit (or observe with sensors) all places (at least or only once)
- Formation planning: Move from A to B in a formation under specified rules





Motion Planning
Required or Desirable Objectives

- ☑ Find a **path** π that moves the robot from initial to goal configurations **never colliding with obstacles** (***admissibility***)

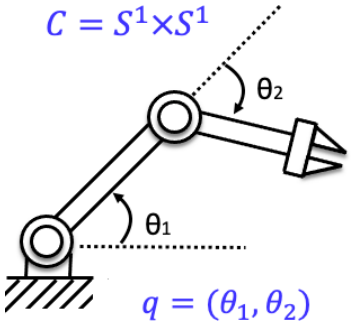
- ☑ Find a **path that complies with robot motion constraints** (***feasibility***)
- ☑ If no feasible + admissible solution exist → Report a *failure* (PSPACE-hard)
- ☐ ***Optimality:*** If more than one solution exists, select the best
- ☐ ***Completeness:*** If at least one feasible + admissible path exists, the path planning algorithm is able to *find it.*

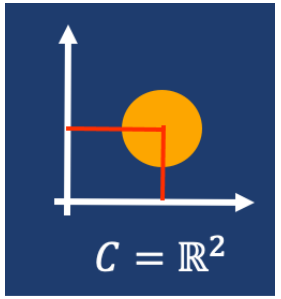
Optimality metrics: **Shortest distance**, **Minimum time**, Minimum curvature, Min effort, Max smoothness, Maximal safety, Min/Max ...

Configuration Space: C-Space

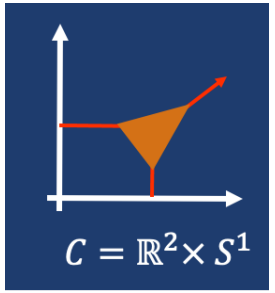
Generalized coordinates: n parameters $\boldsymbol{q} = (q_1, q_2, \ldots, q_n)$ that are sufficient to <u>uniquely</u> describe <u>system configuration</u> relative to some reference (frame, configuration)

Configuration space (C-space, $\mathcal{C}$): n-dimensional space identified by the generalized coordinates, defining the set of <u>possible robot's configurations</u>



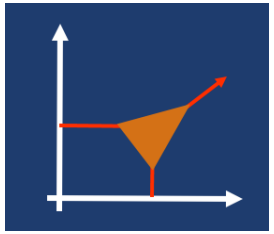


$q = (x, y)$



$q = (x, y, \theta)$

Workspace: set $\mathcal{W}$ of all configurations that the robot, based on its physical structure, can <u>**feasibly reach**</u> in the embedding environment to perform its work

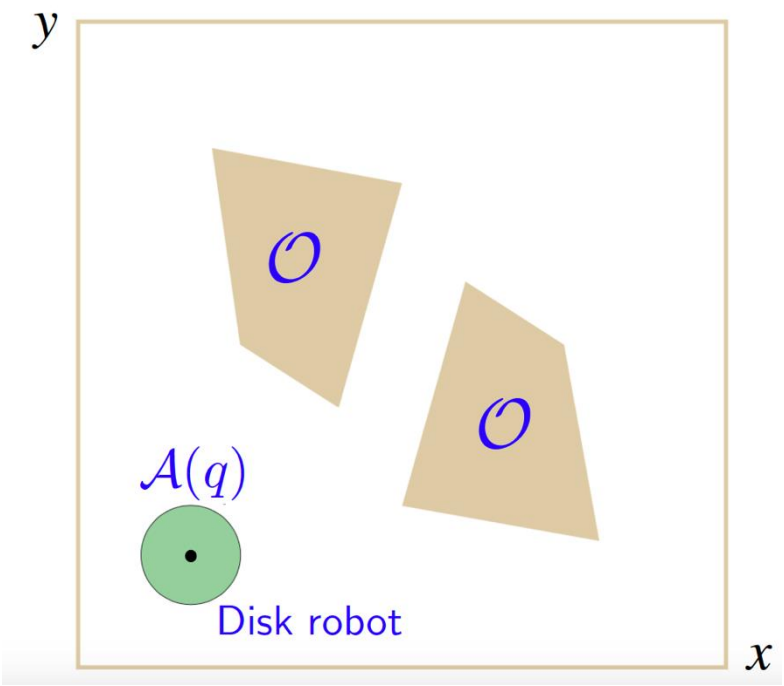


$$\mathcal{W} = \mathbb{R}^2 \times S^1$$

$$(x, y) \quad \theta$$

Motion Planning

Problem Formulation: Free and Occluded Space



❖ **Obstacle region**: $\mathcal{O}$, $\mathcal{O} \subseteq \mathcal{W}$

set of all points in the workspace that lie in one or more **obstacles**

- In general, we will safely assume that $\mathcal{W} \subseteq \mathbb{R}^2$ or $\mathcal{W} \subseteq \mathbb{R}^3$

❖ **Robot**: $\mathcal{A}$, $\mathcal{A} \subseteq \mathcal{W}$

set of all points in the workspace that are covered/occupied by the robot

- $\mathcal{A}(q)$ is the set of workspace points occupied by the robot when in configuration q

Motion Planning

Problem Formulation: Free and Occluded Space

- **Obstacle region for robot in *configuration space*:** $\mathcal{C}_{obs} \subseteq \mathcal{C}$

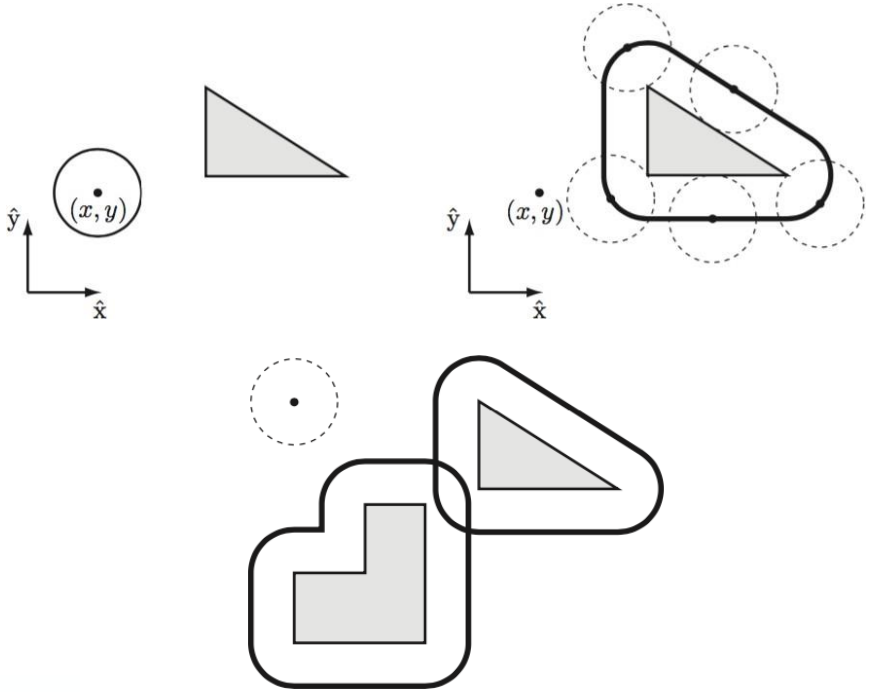
  set of all configurations q at which the *transformed* robot $\mathcal{A}(q)$ would intersect the obstacle region $\mathcal{O}$

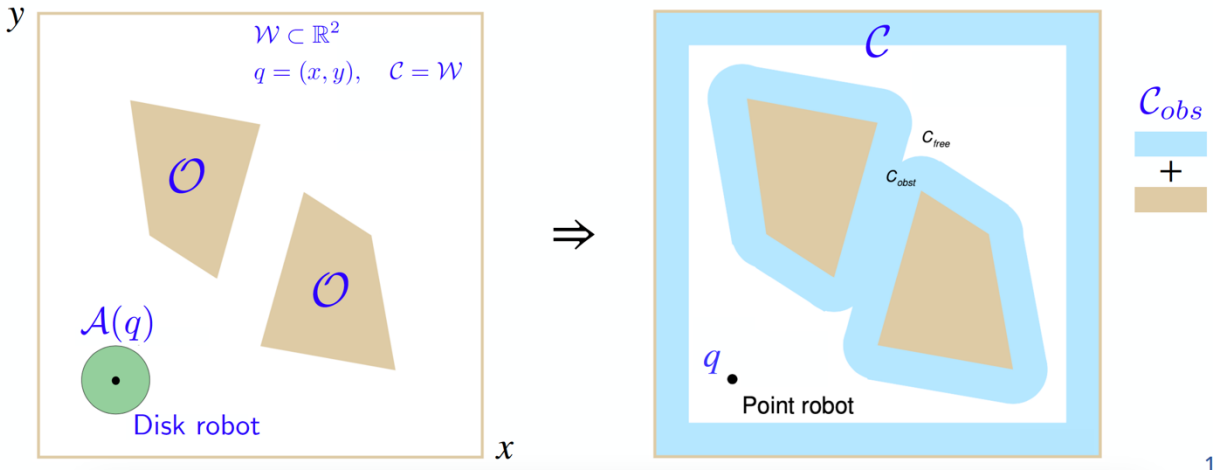
$$\mathcal{C}_{obs} = \{q \in \mathcal{C} \mid \mathcal{A}(q) \cap \mathcal{O} \neq \emptyset\}$$

$\mathcal{C}_{obs}$ obtained by enlarging the obstacles by the disk of radius

By applying Minkowski sum: $\mathcal{O} \oplus \mathcal{A} = \{x + y \mid x \in \mathcal{O}, y \in \mathcal{A}\}$

- **Free space region for robot in *configuration space*:** $\mathcal{C}_{free} = \mathcal{C} \setminus \mathcal{C}_{obs}$

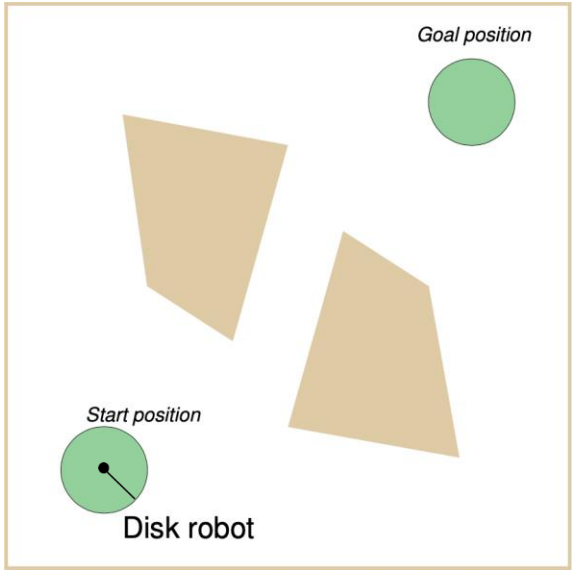




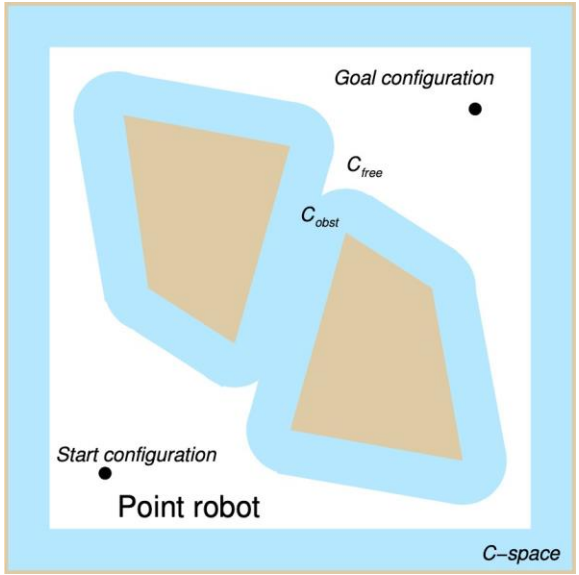
Motion Planning

Problem Formulation: Abstracting Robot as a Point

For the purpose of **motion planning**, once geometry of robot and of environment, including the obstacles, are represented in **configuration space**, the robot is conveniently reduced to a **point**, *since its geometry is already accounted in* C_{free}



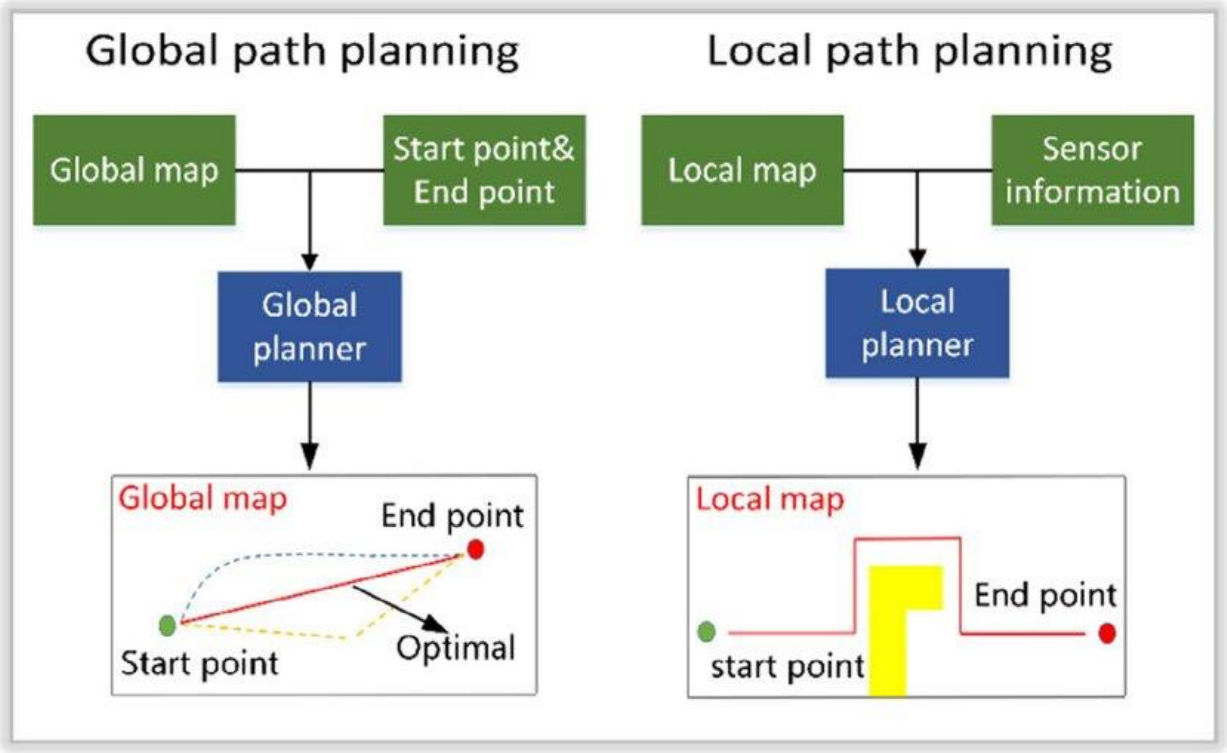
Motion planning problem in geometrical representation of $\mathcal{W}$



Motion planning problem in $\mathcal{C}$-space representation

Global and Local Planning

- Global Planning: Planning in full C-space with **a known map**.
- Local Planning: Planning in robot's limited perceptual space, with dynamics and uncertainty.



(*global*) path and (*local*) obstacle avoidance

Constraints (may consider)

- Geometric (collision).
- Kinematic (nonholonomic).
- Dynamic (acceleration, velocity).

Global Path Planning

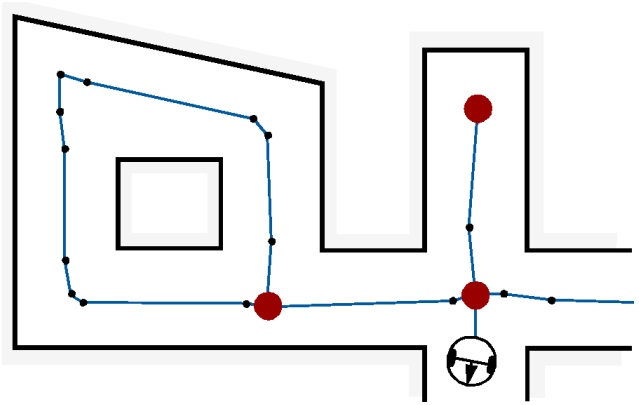
- Assumption: there exists a good enough map of the environment for navigation.
  - Topological, metric or a mixture between both.

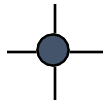
- Steps:
  - Representation of the environment by a road-map (graph), cells or a potential field.
  - The resulting discrete locations or cells allow then to use standard planning algorithms.

- Methods:
  - Visibility Graph
  - Voronoi Diagram
  - Cell Decomposition -> Connectivity Graph
  - Potential Field

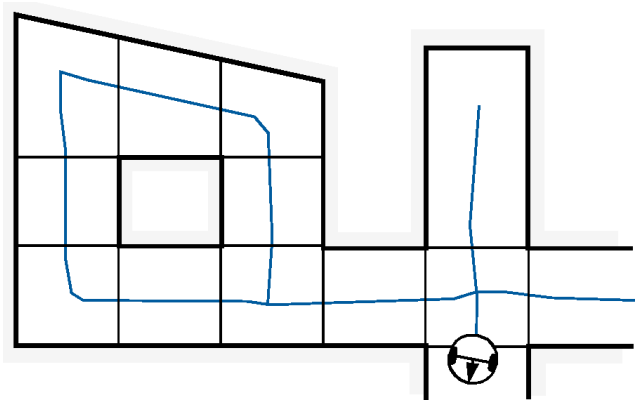
Global Path Planning

1. Road-map based methods
   - Identify a set of routes within the free space



- Where to put the nodes?
- Topology-based:
  - at distinctive locations 
- Metric-based:
  - where features disappear or get visible

2. Cell based methods
   - Discriminate between free and occupied cells

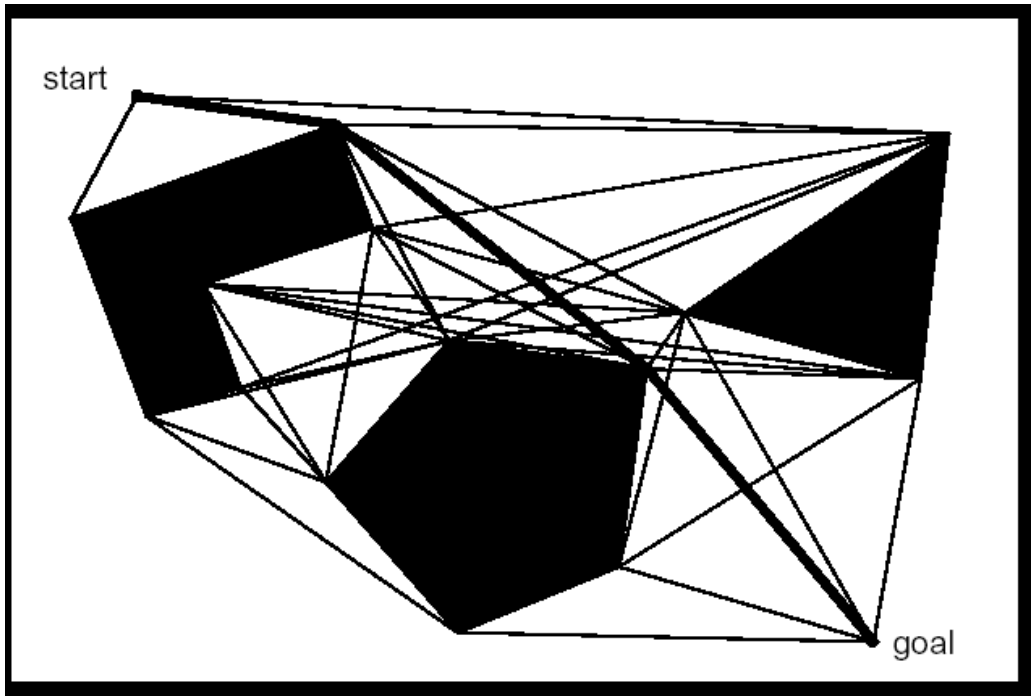


- Where to put the cell boundaries?
- Topology- and metric-based:
  - where features disappear or get visible

3. Potential Field
   - Imposing a mathematical function over the space

Road-Map based Path Planning

Visibility Graph

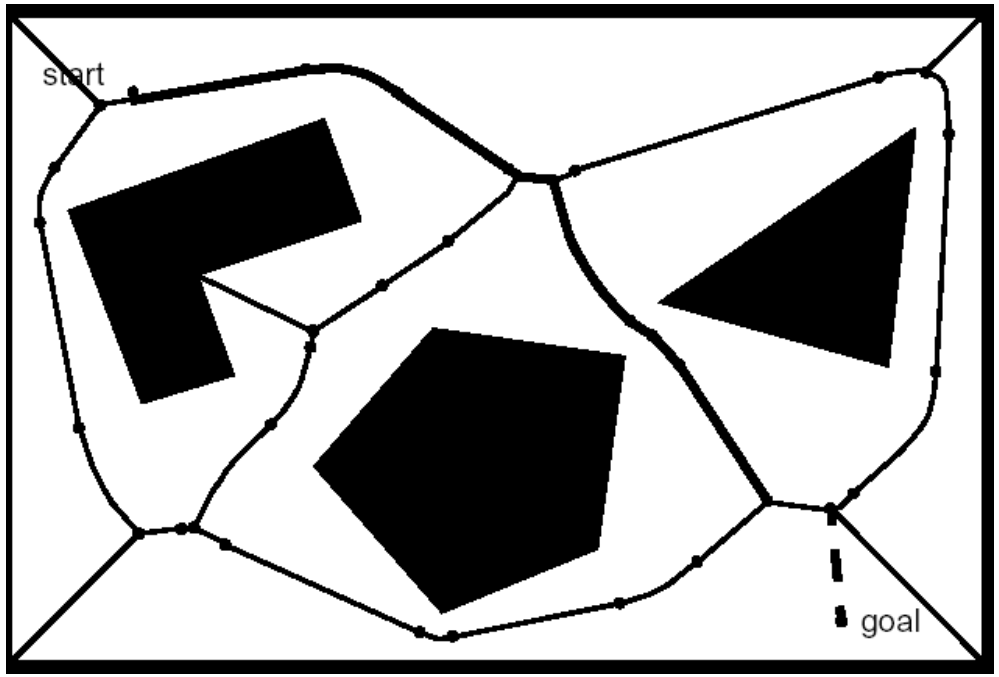


- Shortest path length in polygonal environments
- Efficient for polygonal obstacles
- Otherwise, grow obstacles to avoid collisions

- **Close to obstacles**: Paths often pass extremely close to obstacles → risky for real robots with size/uncertainty.
- **Sensitive to environment complexity**: Becomes computationally expensive with many vertices.
- **Not suitable for nonholonomic constraints**: Assumes point robots with holonomic motion.

Road-Map based Path Planning

Voronoi Diagram



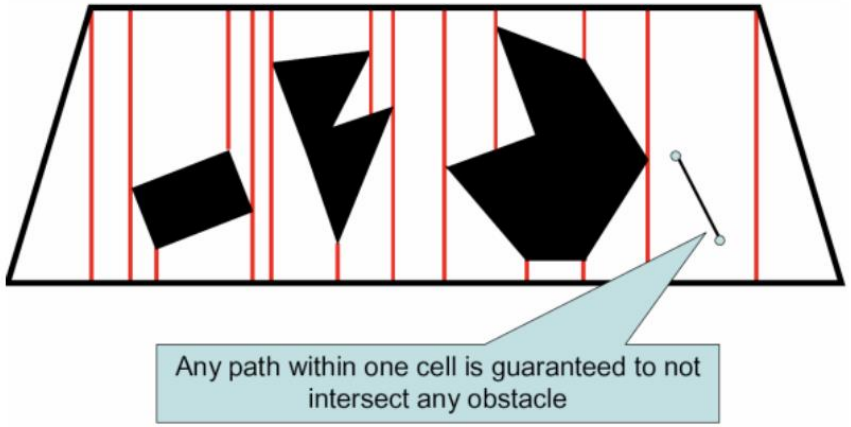
Maximizes clearance: Keeps robot as far from obstacles as possible → safer for real-world execution.

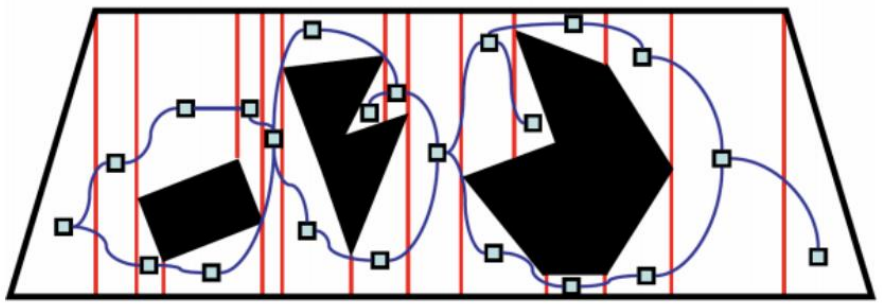
- **Not optimal paths**: Often significantly longer than shortest possible route.
- **Sensitive to small obstacles**: Voronoi edges can be cluttered or irregular.
- **Harder to compute in complex spaces**: Particularly in high-dimensional C-spaces or with curved boundaries.

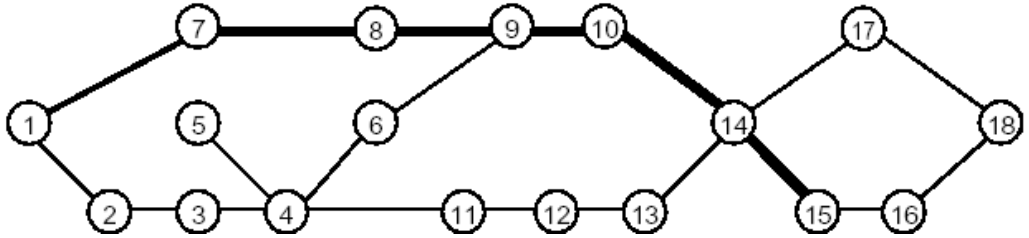
Cell based Path Planning

Cell Decomposition

- Divide space into simple, connected regions called cells
- Determine which open sells are adjacent and construct a connectivity graph (each node is a cell, and edges represent adjacency)
- Find cells in which the initial and goal configuration (state) lie and search for a path in the connectivity graph to join them.
- From the sequence of cells found with an appropriate search algorithm, compute a path within each cell, e.g. passing through the midpoints of cell boundaries or by sequence of wall following movements.

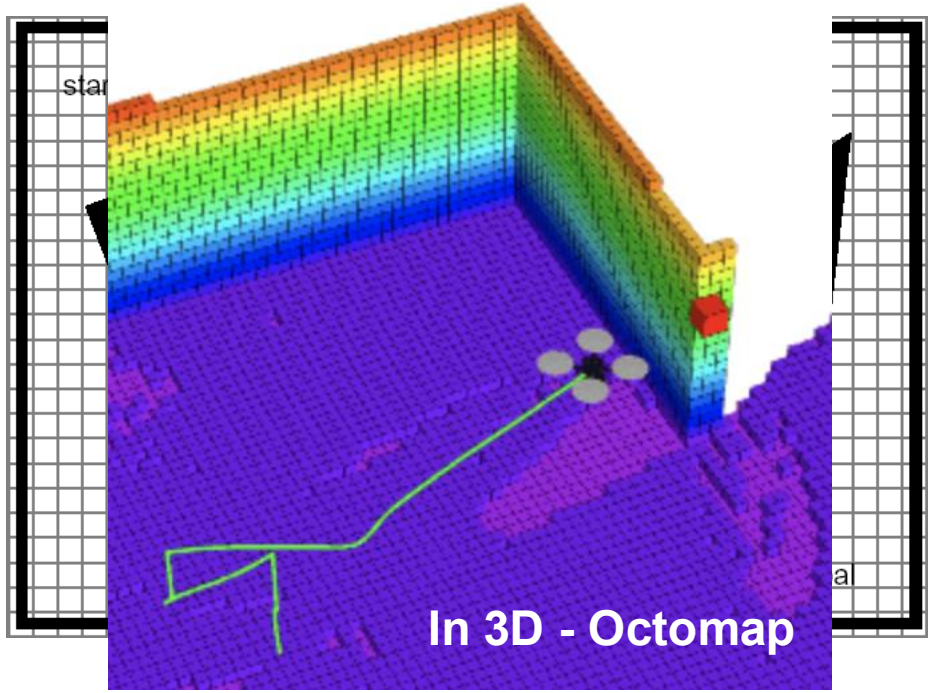


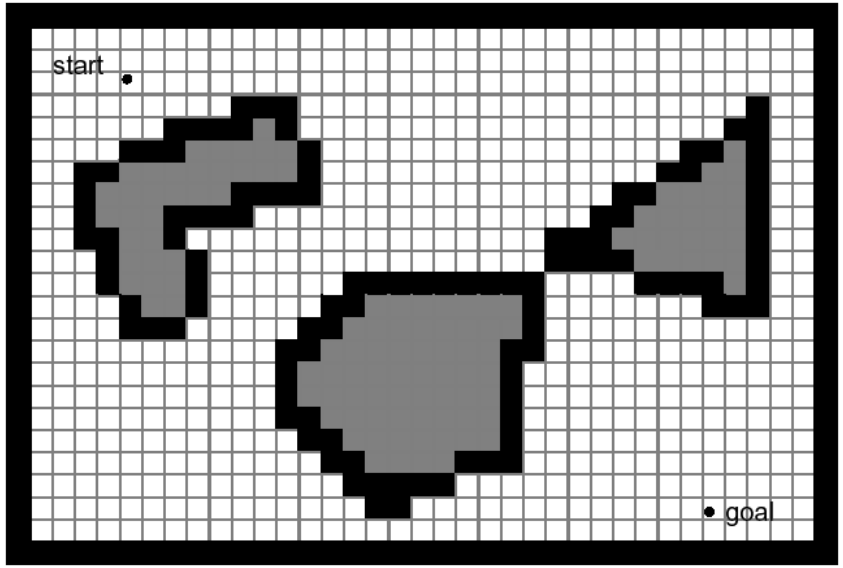




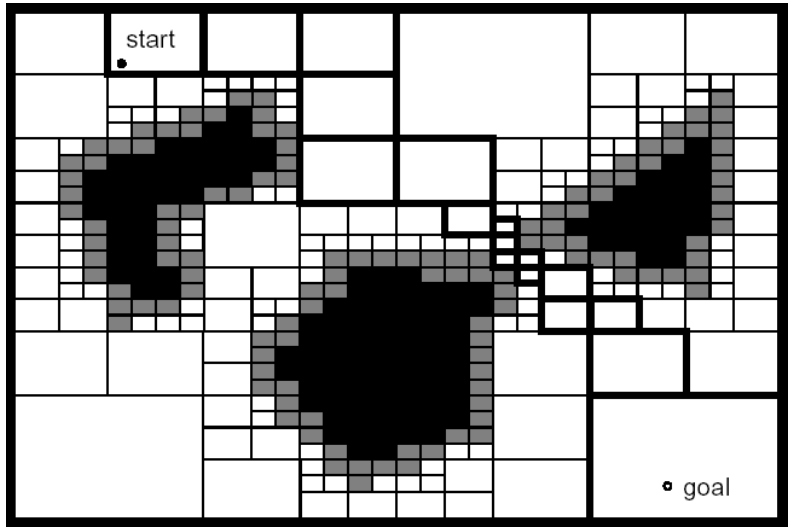
Cell based Path Planning

Cell Decomposition





Approximate Cell Decomposition
(occupancy grids)



Adaptive Cell Decomposition

Converts a continuous problem into a discrete one by discretizing the space first to cells, and then searching

Cell based Path Planning

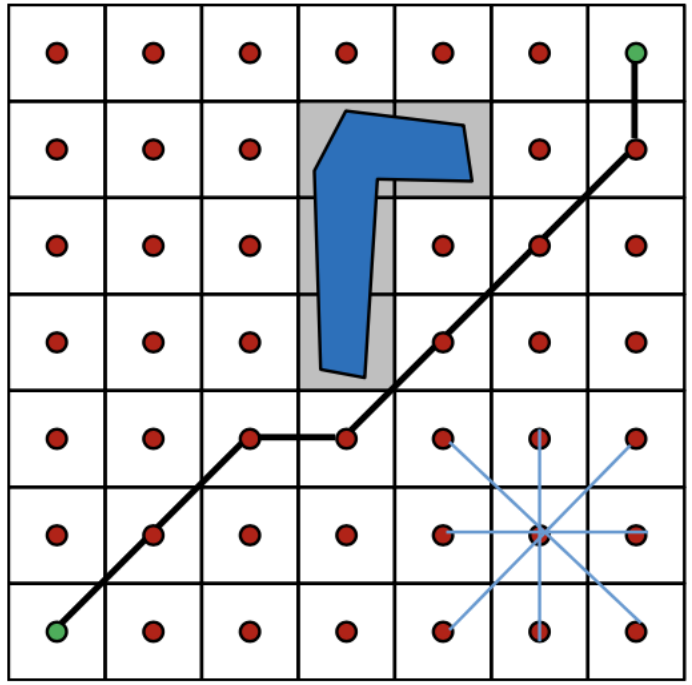
Path Search

Motion plan can be found by searching the grid for the best (e.g., shortest) path that joins initial and goal cells

Navigation execution: For each cell in the path sequence, compute a navigation path within each cell for moving into the cell that follows in the path

Search algorithms:

- Dijkstra's
- A∗
- D* (for online replanning, dynamic environments)

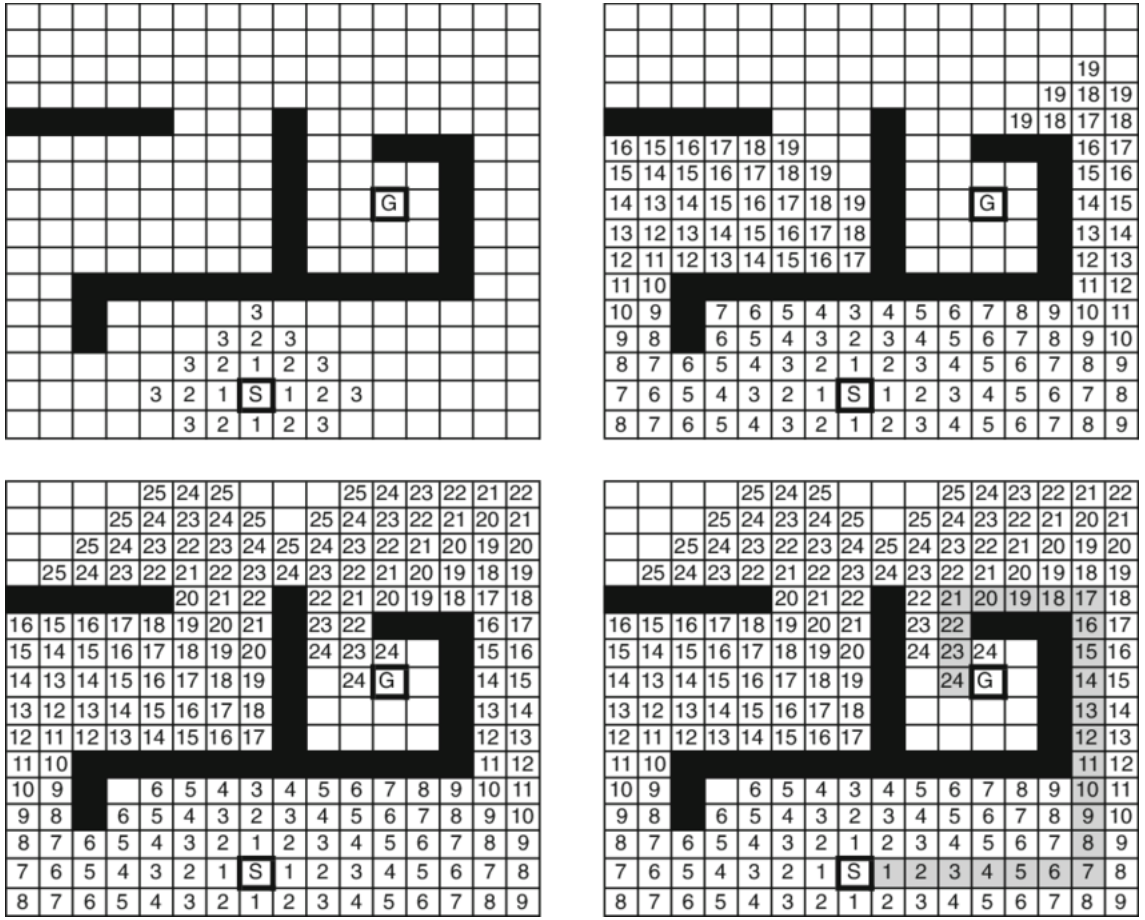


Dijkstra’s Algorithm

Dijkstra’s Algorithm: Finds the **shortest path** from a source node to all other nodes using **uniform cost**.

- Explores all reachable nodes in order of increasing cumulative cost.
- No heuristic → pure cost-based expansion

$$\mathrm{Cost}(q) = \mathrm{Cost}(q_{\mathrm{parent}}) + \mathrm{EdgeCost}(q_{\mathrm{parent}}, q)$$



+ Guarantees optimal path
+ Simple and deterministic

- Inefficient in large spaces (no focus toward the goal)

A∗ Algorithm

A*

- Improved Dijkstra's algorithm
- uses an additional heuristics (e.g., distance to the goal) to guide its search for better efficiency, because we don't need to visit every node.

Combines cumulative cost with heuristic:

$$f(q) = g(q) + h(q)$$

- $g(q)$: known cost from start to q
- $h(q)$: heuristic estimate from q to goal (e.g., Euclidean or Manhattan)

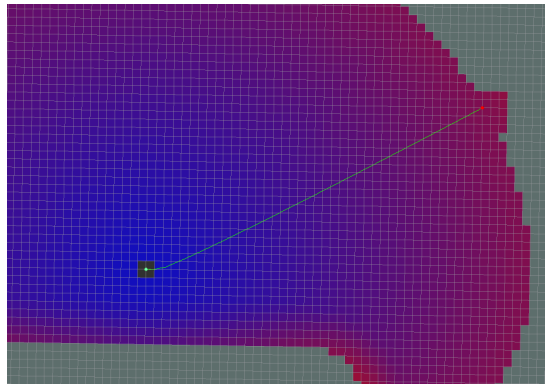
+ Efficient: Focused exploration toward goal

- Depends on quality of heuristic
- Not inherently dynamic (unlike D*)

Dijkstra's algorithm

Iteration list:

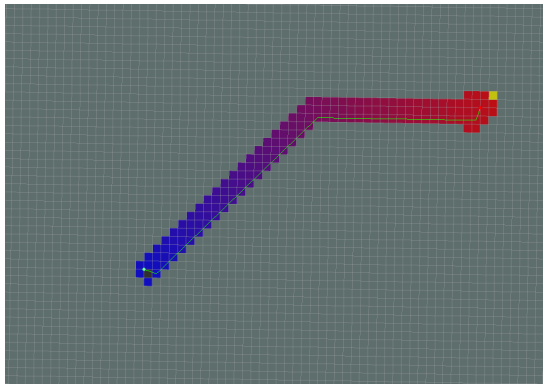
```
[0,  1, -1, 25, 28, 31, 35, 40, 45, 50, 54]
[2,  3, -1, 23, 26, 29, 32, 36, 41, 46, 51]
[4,  5, -1, 20, -1, -1, 37, 42, -1, 52, 55]
[6,  7, -1, 17, 21, 24, -1, 47, 53, 56, 57]
[8,  9, 11, 14, -1, 27, -1, 43, 48, -1, 58]
[10,12,15, 18, -1, 30, 33, 38, -1, 60, 59]
[13,16,19, 22, -1, 34, 39, 44, 49, -1, 61]
```



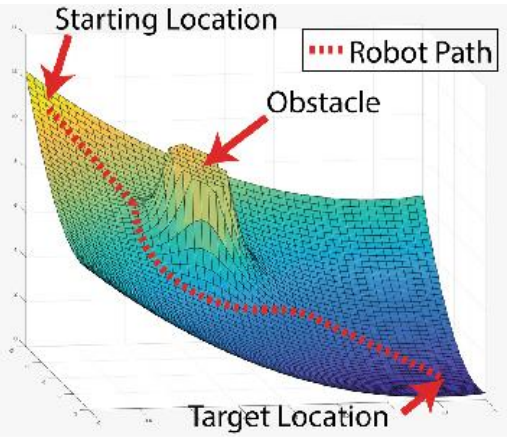
A*

Iteration list:

```
[0,  1, -1, -1, -1, -1, -1, -1, -1, -1, -1]
[2,  3, -1, 34, 35, 36, 37,38, 40, 42, 44]
[4,  5, -1, 31, -1, -1, 39, 41, -1, 45, 47]
[6,  7, -1, 20, 21, 22, -1, 43, 46, 48, 49]
[8,  9, 11, 14, -1, 23, -1, 32, 33, -1, 50]
[10,12,15, 17, -1, 24, 25, 27, -1, -1, 51]
[13,16,18, 19, -1, 26, 28, 29, 30, -1, 52]
```



Potential Field based Path Planning



Robot is treated as a point under the influence of an artificial potential field

- Generated robot movement is similar to a ball rolling down the hill
- Goal generates attractive force
- Obstacle are repulsive forces

- **Total Potential:**

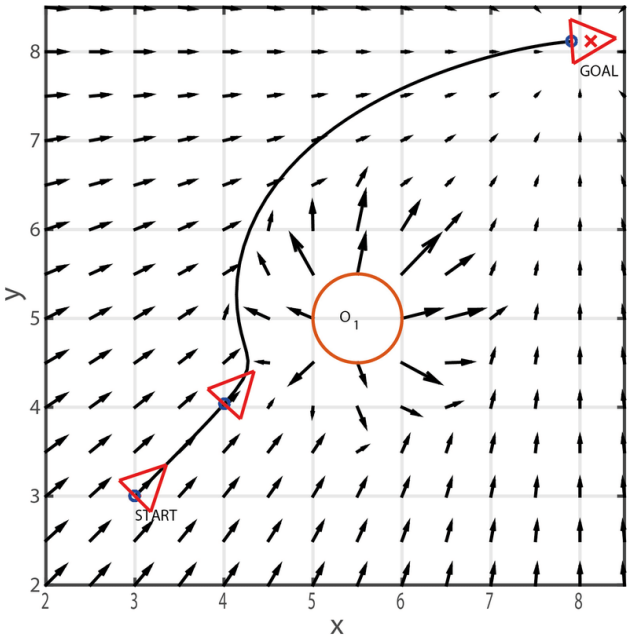
$$U(q) = U_{\text{att}}(q) + U_{\text{rep}}(q)$$

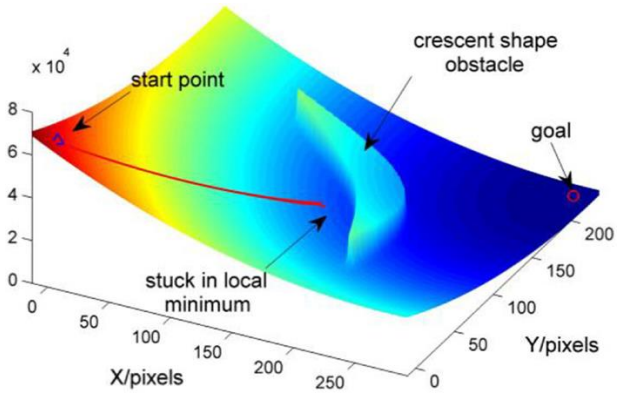
- **Attractive Potential (to goal q_g):**

$$U_{\text{att}}(q) = \frac{1}{2} k_{\text{att}} \|q - q_g\|^2$$

- **Repulsive Potential (from obstacle q_o):**

$$U_{\text{rep}}(q) = \begin{cases} \frac{1}{2} k_{\text{rep}} \left(\frac{1}{\rho(q)} - \frac{1}{\rho_0} \right)^2 & \text{if } \rho(q) \leq \rho_0 \\ 0 & \text{otherwise} \end{cases}$$





Path: follows gradient descent: $\dot{q} = -\nabla U(q)$

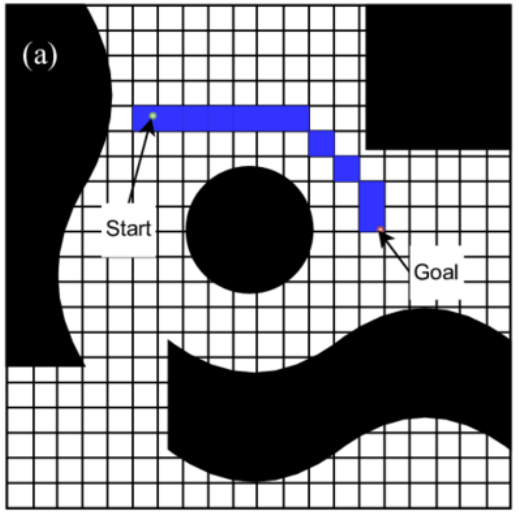
Key Issues:

- Local Minima: Robot can get stuck in non-goal minima.
- Oscillations: Especially in narrow passages.

Sampling based Path Planning

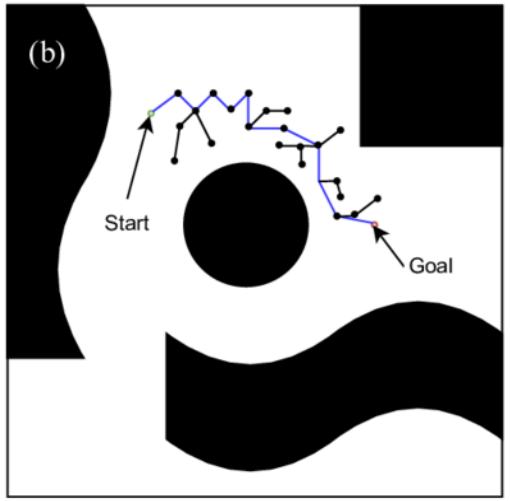
Cell/Grid-based search methods:

high computational complexity, only appropriate for low-dimensional C-space or state space



Sampling based methods

relies on a random or deterministic function to choose a sample from the C-space or state space



Two major types of sampling methods

- Rapidly-exploring Random Tree (RRT)
- Probabilistic Roadmap (PRM)

Rapidly-exploring Random Tree (RRT / RRT*)

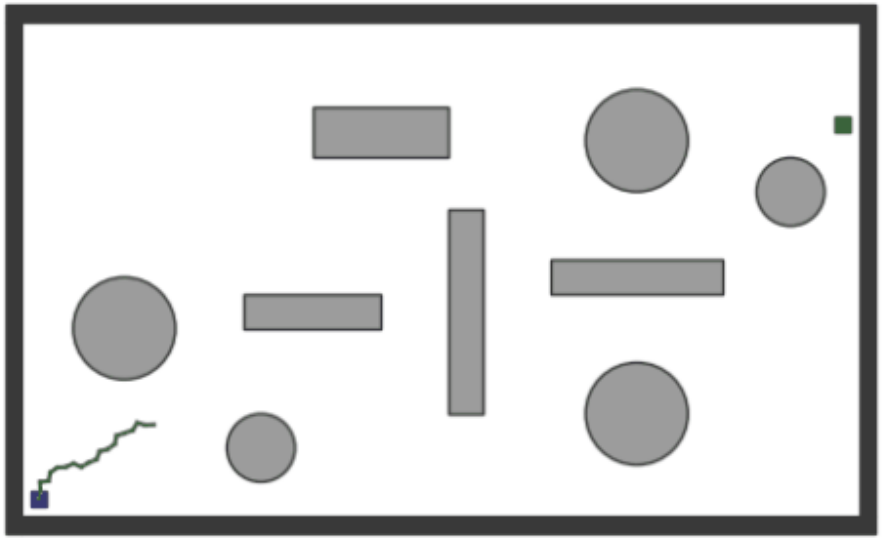
Rapidly-exploring Random Tree (RRT):

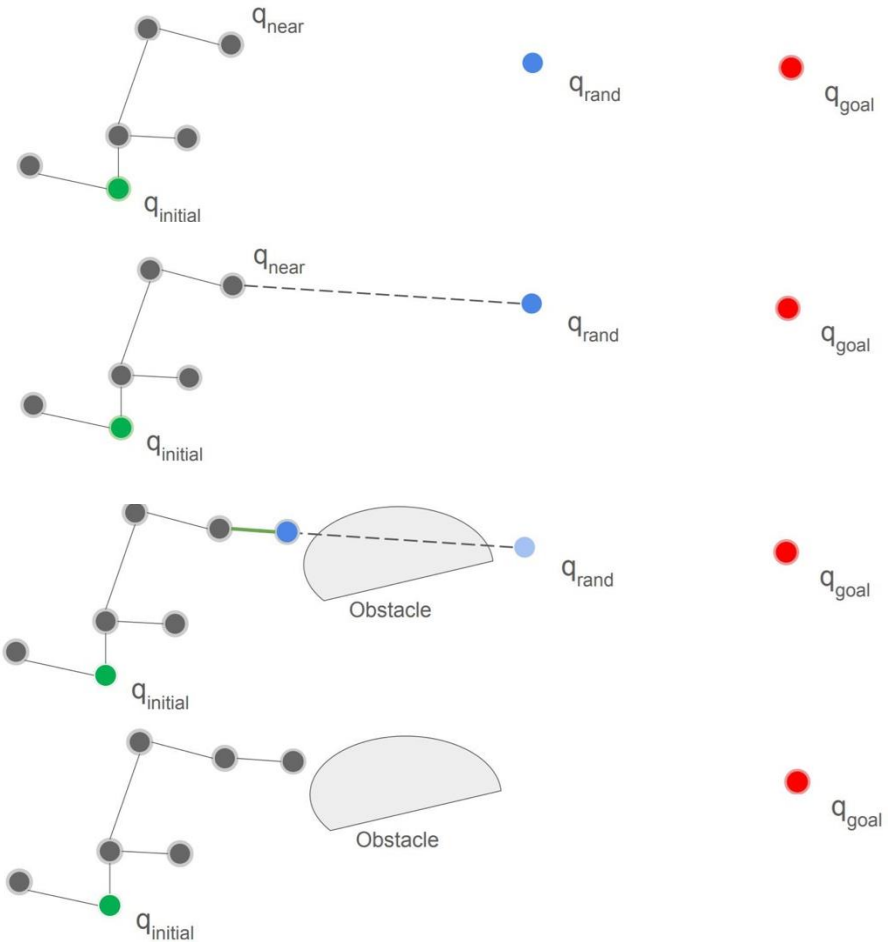
incrementally grow a random tree from the start configuration to explore large, high-dimensional C-spaces

The explored territory is marked by a tree rooted from the initial configuration (start).

Algorithm:

1. Sample a random configuration q_{rand}
2. Find nearest node q_{near} in the tree
3. Extend tree toward q_{rand} by step-size Δq
4. Add the new node if path is collision-free
5. Repeat until goal is reached or time limit hit



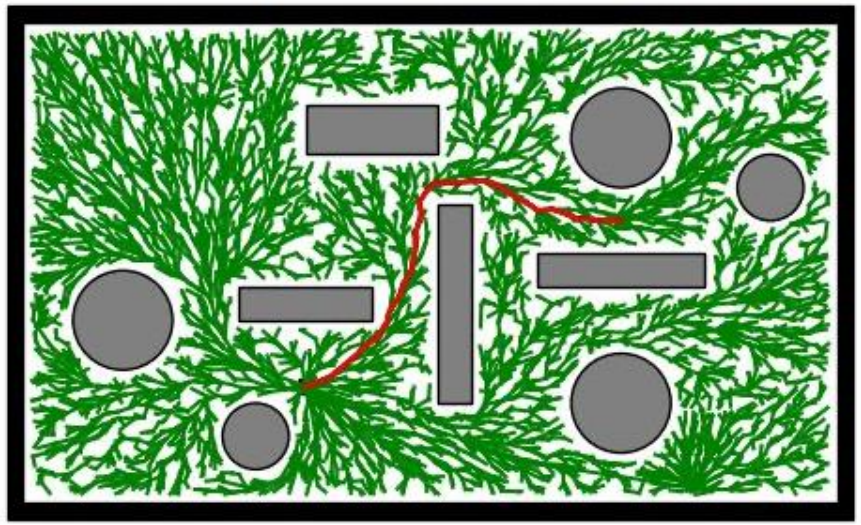


Rapidly-exploring Random Tree (RRT / RRT*)

RRT* (Optimal Version):

Adds a rewiring step to minimize path cost:

- Connects new node to the best nearby parent
- Rewires neighbors if a better path through new node exists
- Favors **lower-cost connections** over proximity



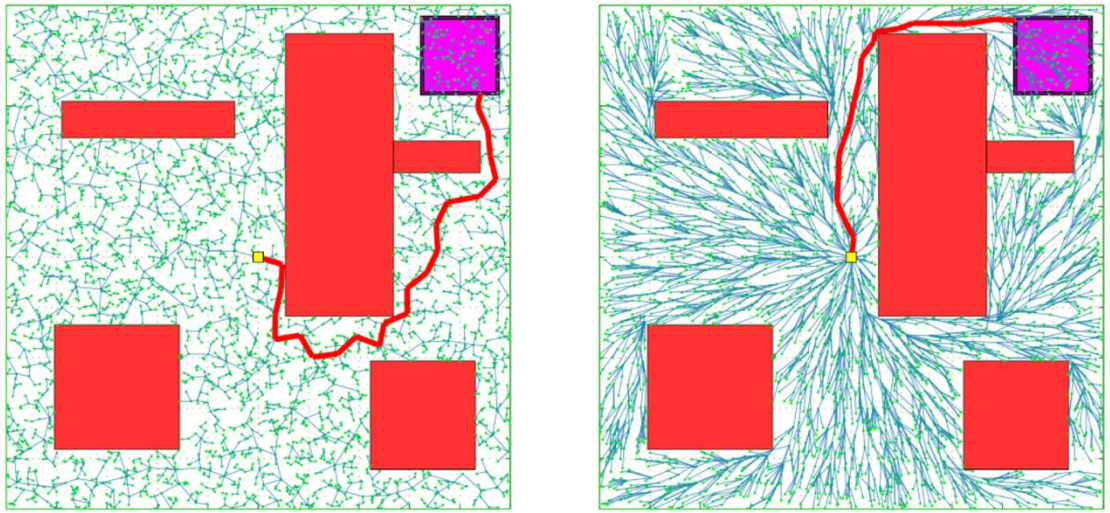
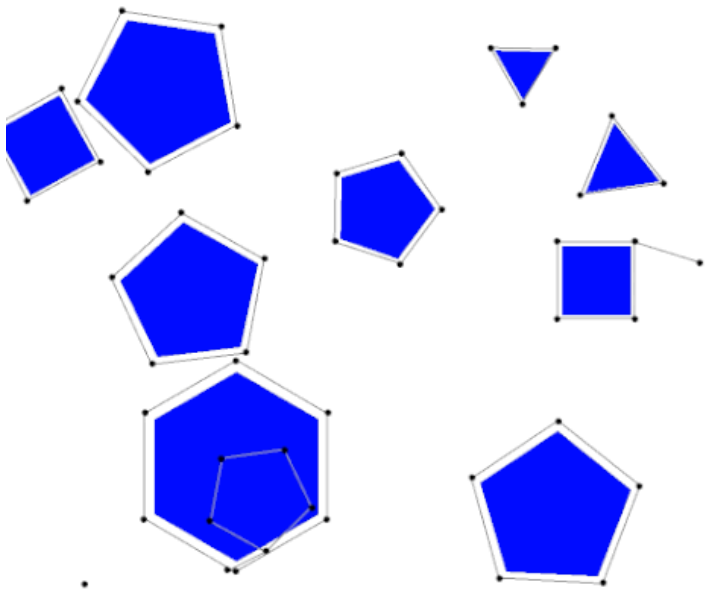


Figure 10.19: (Left) The tree generated by an RRT after 5,000 nodes. The goal region is the square at the top right corner, and the shortest path is indicated. (Right) The tree generated by RRT* after 5,000 nodes. Figure from [67] used with permission.

+ Fast in high-dimensional spaces
+ Works well for differential constraints
+ RRT*: Optimal with enough samples

- RRT: Not optimal
- RRT*: Slower convergence than RRT
- Sensitive to sampling strategy & step size

Probabilistic Roadmap (PRM)

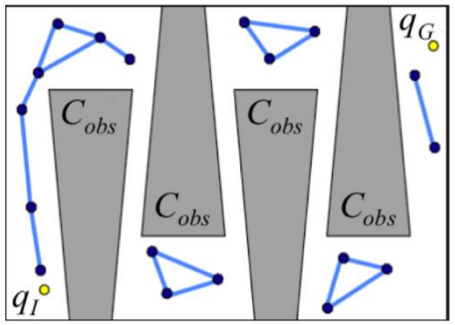


Probabilistic Roadmap (PRM):

Construct a roadmap of free space by randomly sampling nodes and connecting nearby nodes to form a graph.

Algorithm with two phases:

1. Learning Phase:
   - Sample random configurations in C_{free}
   - Connect each node to nearby nodes achievable with kinematic constraints
   - Store connections if local paths are collision-free
2. Query Phase:
   - Connect start and goal to the roadmap
   - Use graph search (e.g., A*) to find a paths



+ Very efficient for multi-query environments
+ Scales well in high DoF problems

- Requires good sampling density and connection radius
- Hard in narrow passages without extra sampling effort
- Neither complete nor optimal

Local Planning
Obstacle Avoidance

Purpose: follows the global path (local target) while avoiding collision with the (dynamic) obstacles observed by sensors and from the local map.

Inputs:

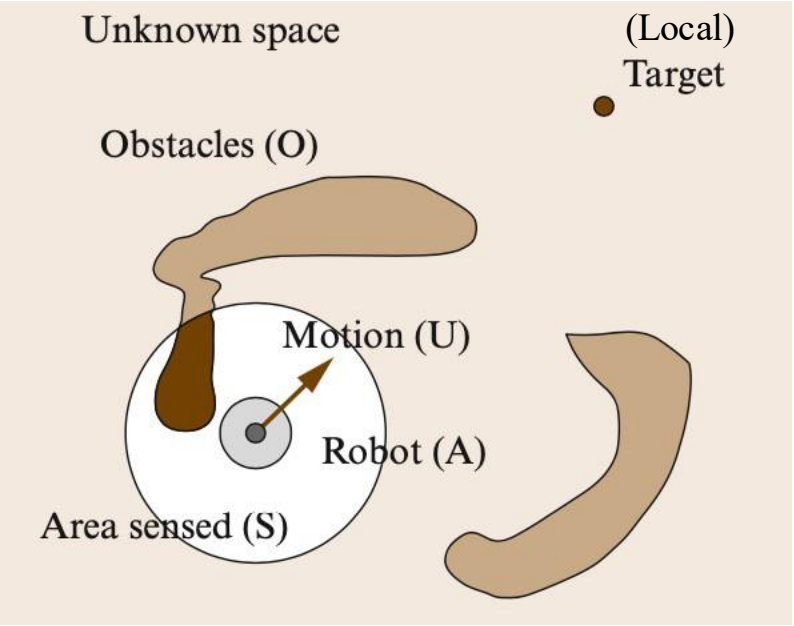
- the global path (or local target)
- the sensor data and their uncertainty
- the local map
- the robot pose, speed and kinematics
- the motion constraints (max speed, turning radius, etc)

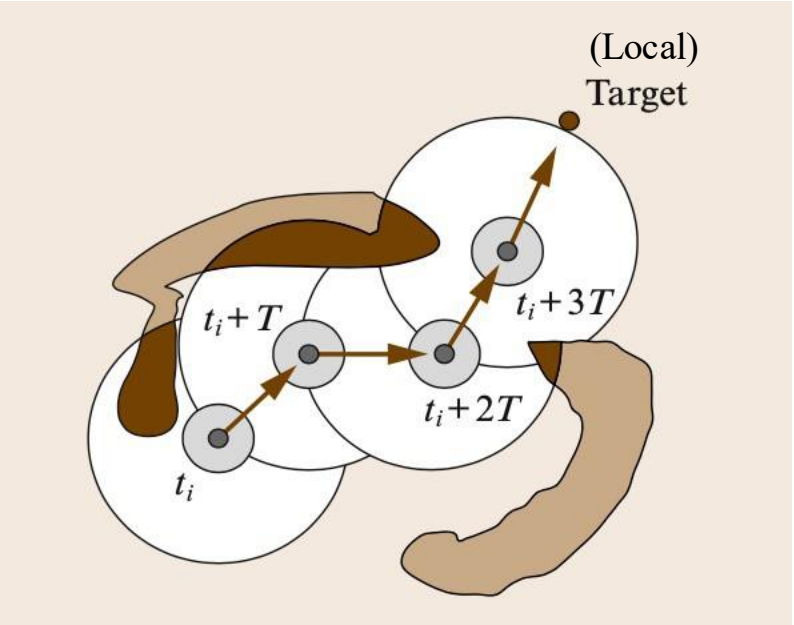
Outputs:

velocity commands (linear and angular) or short trajectory segments (e.g., for control)

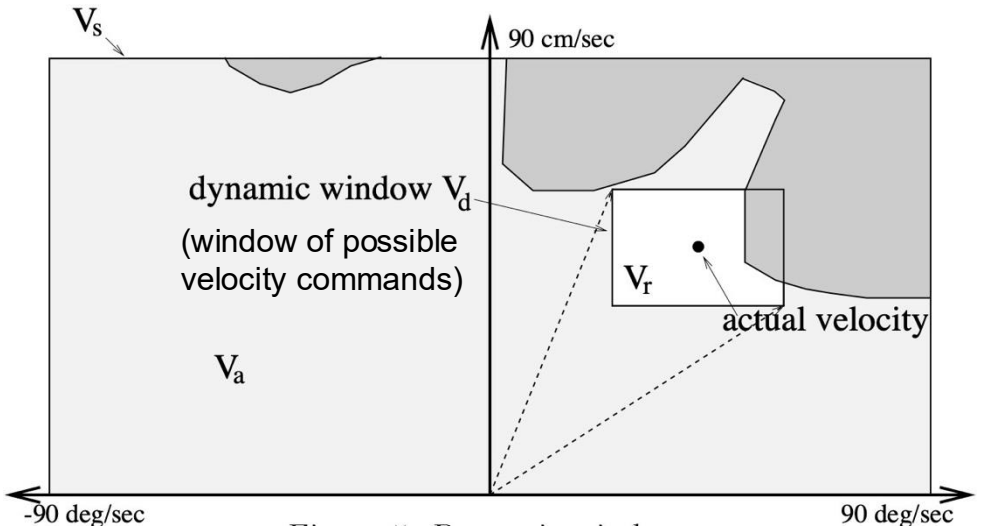
Methods:

- Potential field, RRT
- Bug 1, Bug 2 algorithms
- Dynamic Window Approach (DWA)
- Timed-Elastic-Band (TEB) method





Dynamic Window Approach



Evaluates a set (window) of velocity commands and simulates short forward trajectories, scoring them based on a metric

Algorithm:

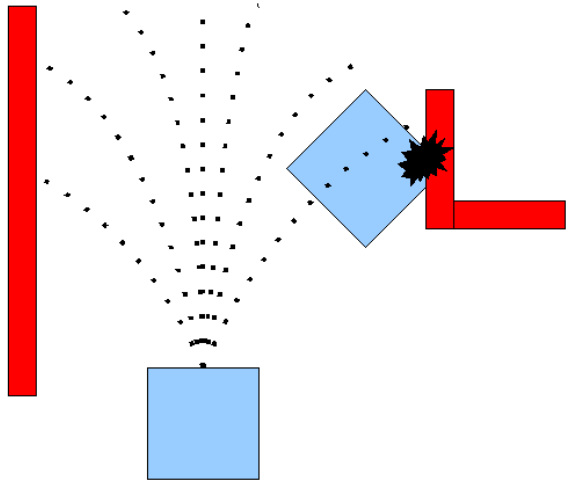
1. Sample velocity space within dynamic constraints
2. Predict motion over a short time horizon (using kinematics!)
3. Score trajectories:

$$G(v,\omega) = \sigma(\alpha \cdot \text{heading}(v,\omega) + \beta \cdot \text{dist}(v,\omega) + \gamma \cdot \text{velocity}(v,\omega))$$

Measure alignment with target: get to goal

Measure Clearance: distance to closest obstacle on circular path → avoid obstacles

Robot velocity: move fast!



4. Select best-scoring velocity command

minimize the travel-time by "driving fast in the right direction"

+ Real-time performance
+ Considers robot dynamics
+ Good for differential-drive robots

- Greedy: May get stuck in local minima
- Relies on accurate short-term prediction

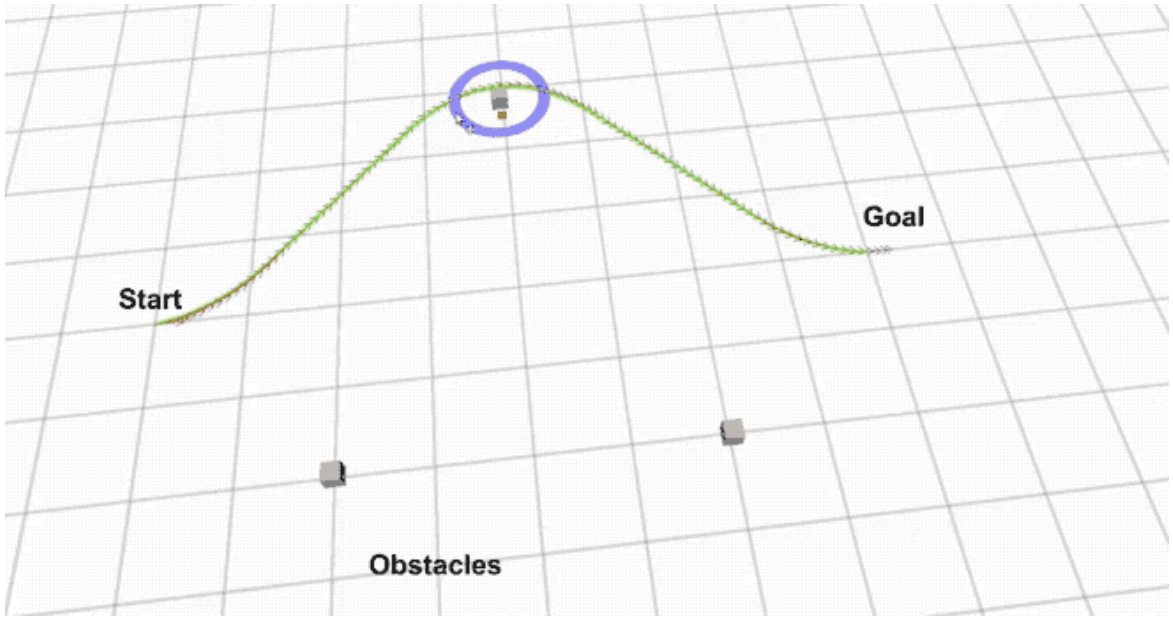
Timed-Elastic-Band (TEB) Method

Formulate **trajectory planning** as an optimization problem over a **band of poses** with time intervals.

Algorithm:
Represent trajectory as a sequence of poses + time stamps to optimize

$$\min_{\text{trajectory}} \quad \text{cost}_{\text{obstacles}} + \text{cost}_{\text{kinematics}} + \text{cost}_{\text{timing}}$$

obstacle avoidance | kinematic constraints | time efficiency



+ Produces smooth, time-parameterized trajectories
+ Handles non-holonomic constraints
+ Good for fast robots, ackermann drive

- Heavier computational load than DWA
- Needs a reasonable initial guesss

Trajectory Planning

Trajectory Planning computes a *time-dependent* path:

$$\mathbf{q}(t) = [x(t), y(t), \theta(t), v(t), \omega(t), \dots]$$

that guides a robot from a start to goal configuration **smoothly and feasibly** over time.

Goals:

- Consider kinematic & dynamic constraints
- Maintain collision avoidance
- Ensure smooth motion (bounded velocity, acceleration, jerk)

Output:

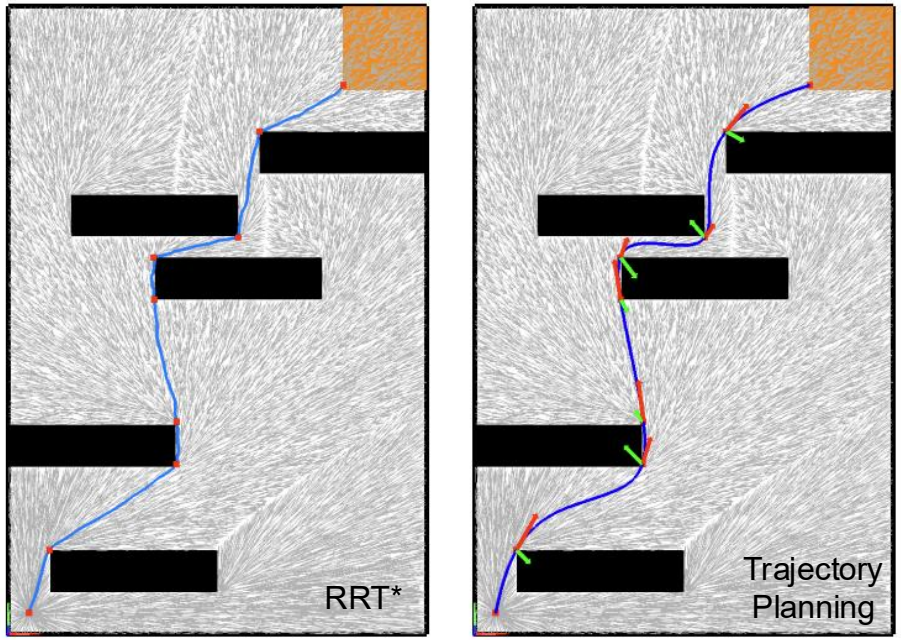
A timed trajectory/sequence of states (poses, velocities) → ready for controller to follow

Trajectory Planning

Approaches

1. Polynomial Trajectory based Methods: Path → polynomial interpolation → trajectory tracking

1) Compute a geometric path (e.g., RRT) as a sequence of waypoints in C_{free}
2) Use polynomials (e.g., cubic, B-splines) to interpolate between waypoints for a smooth curve
3) Add timing → define velocity, acceleration, etc.



2. Optimization based Methods

1) Compute a rough path (e.g., from RRT/PRM)
2) Formulate a constrained optimization problem to refine the path, with constraints
3) Minimize cost: time, smoothness (jerk), curvature, collision, …..

Algorithms:

- CHOMP: Gradient-based trajectory optimization.
- STOMP: Sampling-based trajectory improvement, better at handling non-differentiable cost functions.
- TrajOpt: Uses sequential convex optimization; fast and suitable for real-time applications.

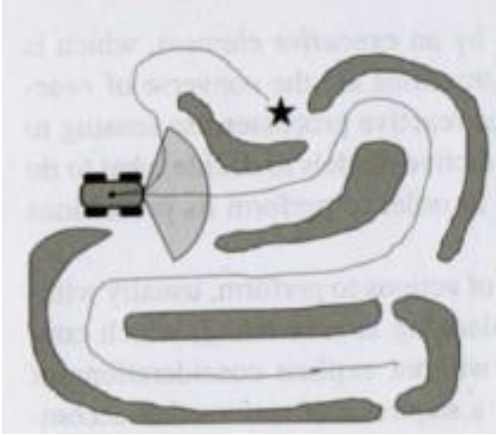
+ Smooth paths and continuous derivatives.
+ Dynamics and constraints included.
+ Fast refinement.

- Needs good initialization.
- Local minima.

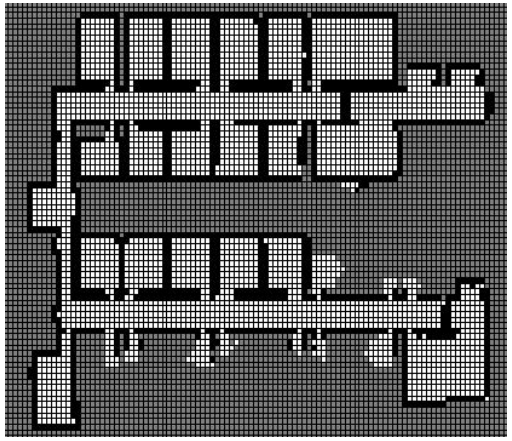
Motion Planning
Practical Considerations

Real-world constraints: computation, sensing, uncertainty.
Real-time planning (e.g., receding horizon).

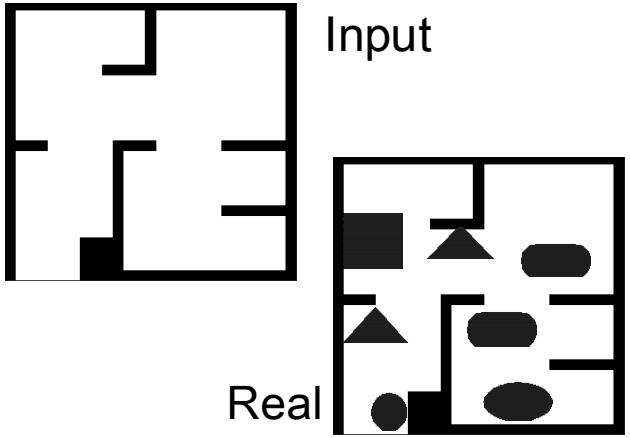
- How far should we look into the future? **Depth of look ahead**
  The farthest, the more *reliable* the plan is and the more *expensive* computation is
- What *spatial resolution* map model? **Accuracy of spatial representation**
  The higher the resolution the higher the computational requirements
- How realistic / reliable is the input map?
- What are the effects of **imprecise or partial models**?



Without planning / looking ahead things can get arbitrarily bad!



An extremely fine-grained representation (occupancy grid) might require heavy computation



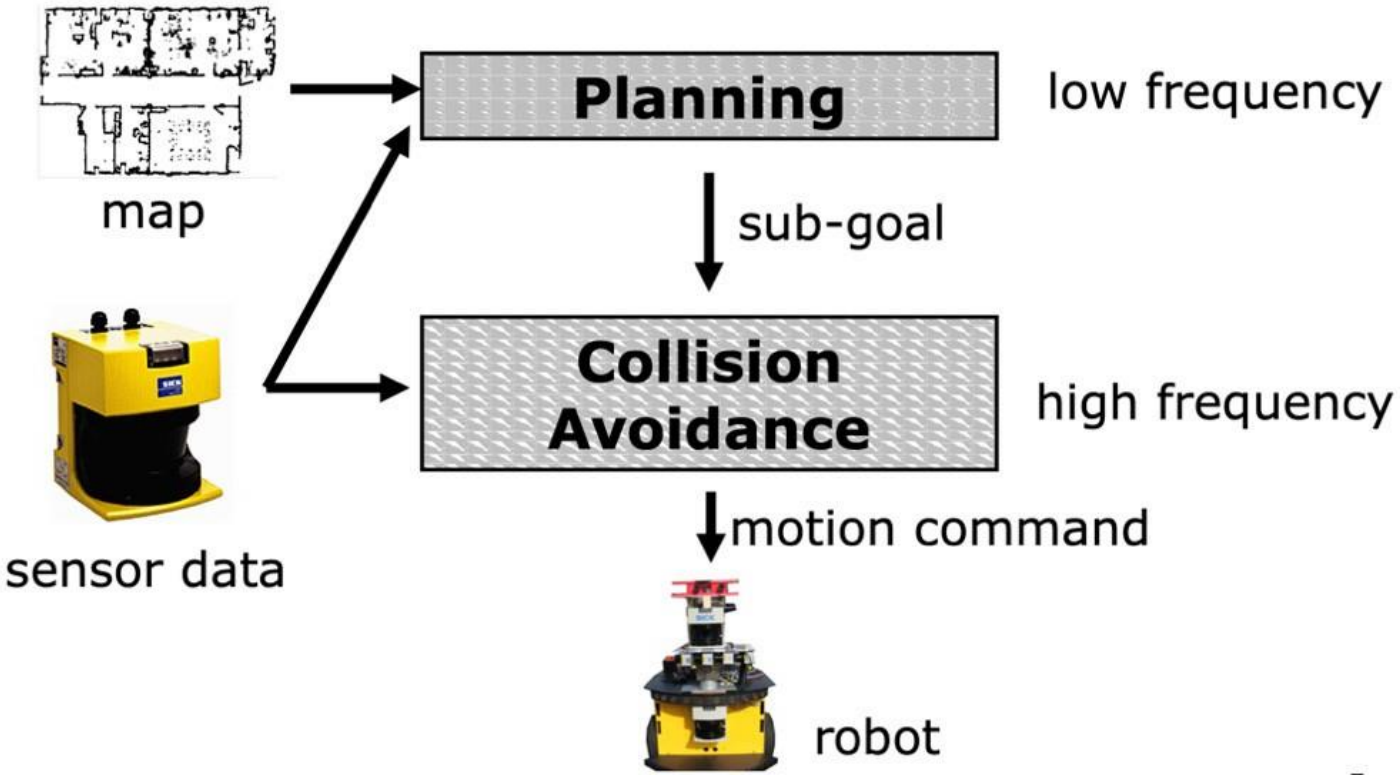
A plan good / feasible in the incomplete input map could easily be not feasible in the real environment

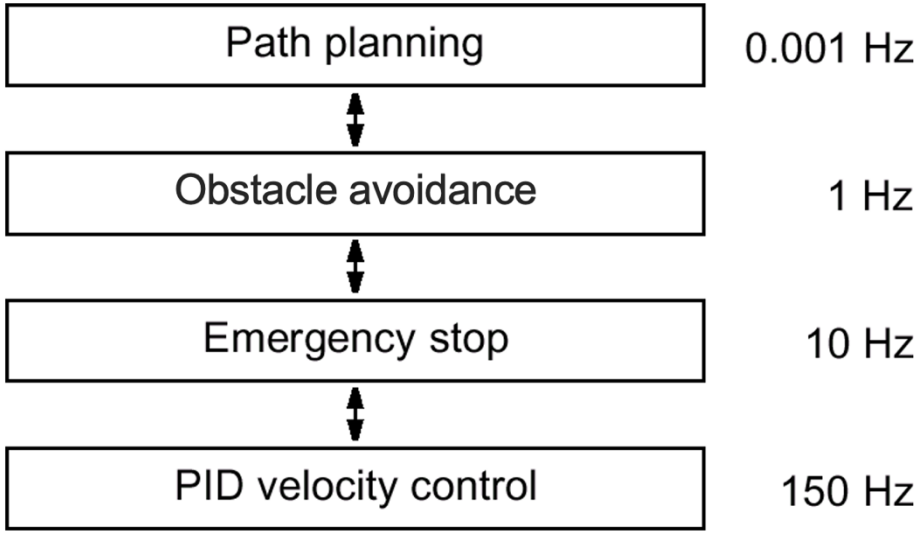
(Strong) Assumptions of Motion Planning

- **Localization**: The robot knows where it is in the world, and is able to maintain a (robust) estimate of this while it moves.
- **Mapping**: Location of the goal is known within the robot's representation of the world.
- **Sensing:** Obstacles need to be sensed with precision
- **Actuation/Control:** The robot is able to smoothly change and adapt its trajectory and <u>velocities</u>

Planning to Navigation

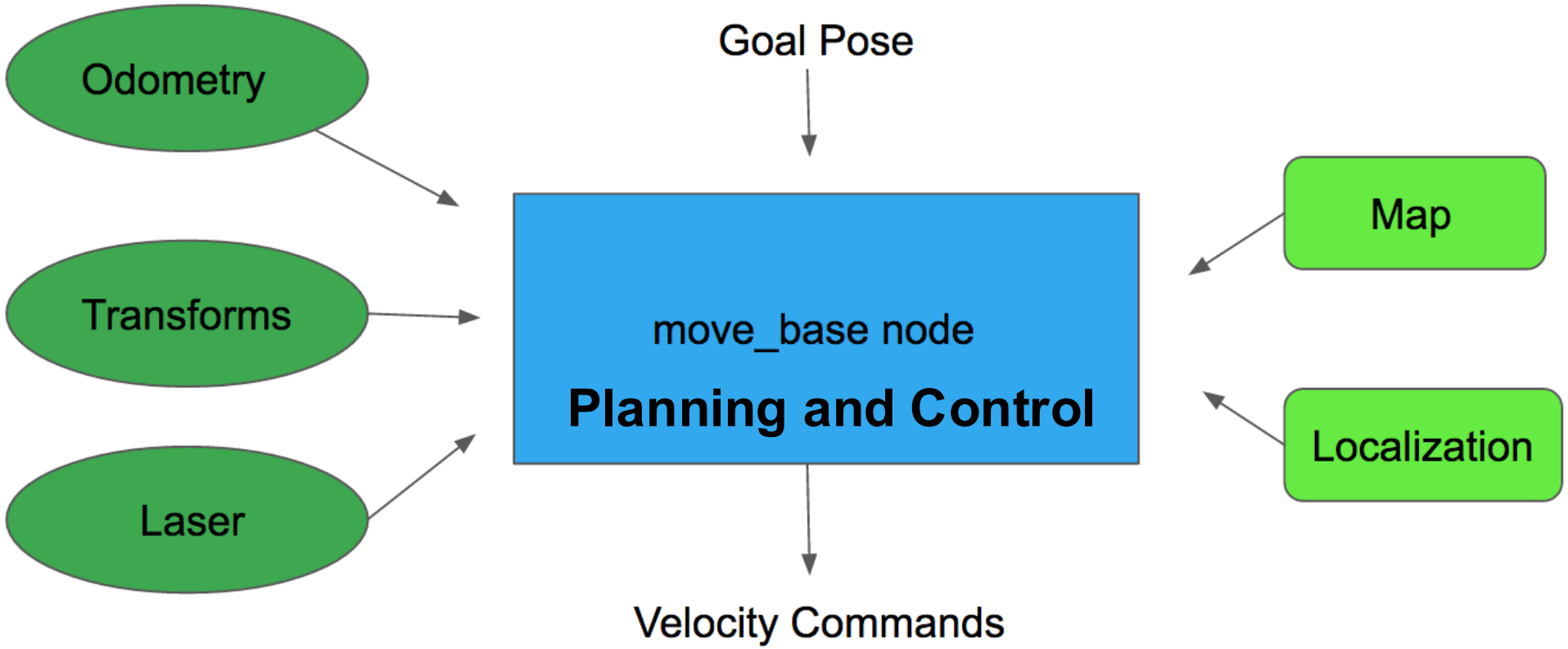
Standard Two-layered Architecture



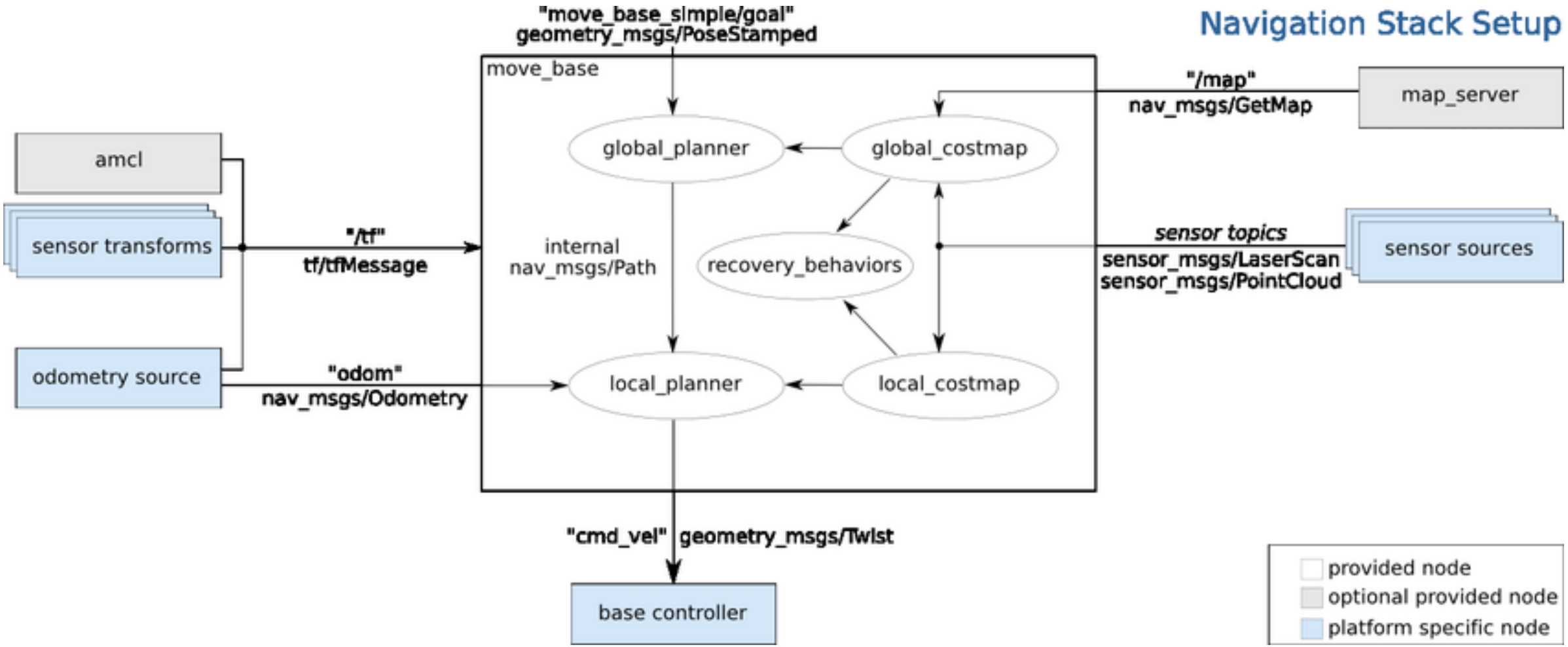


ROS Can Help

Navigation Stack



ROS Navigation Stack



More details:
https://wiki.ros.org/navigation

ROS Navigation Stack

move_base: coordinates all the path planning system

Input: a goal pose
Output: necessary velocity commands moving the robot from a pose to the specified goal pose

- **Global Planner:** A* and Dijkstra-based global path planner to calculate a safe path to reach the specified goal using the global costmap
- **Local Planner:**
  - Once the global planner has calculated a path for the robot, this is sent to the local planner. The local planner, then, will execute this path, breaking it into smaller (local) parts.
  - Trajectory Rollout and Dynamic Window local planner over a local costmap to provide velocity commands controlling the robot.
- **Costmaps:** maps that represent which points of the map are safe for the robot to be in, and which ones are not. There are 2 types of costmaps:
  - Global costmap: built using the map
  - Local costmap: built from the robot's sensor readings.
- **Recovery Behaviors:** methods in case the robot gets stuck, with 2 different recovery behaviors:
  - rotate recovery
  - clear costmap

ROS Packages

ROS Navigation Stack:

- Dijkstra
- A*
- Dynamic Window
- Timed-Elastic-Band (TEB)

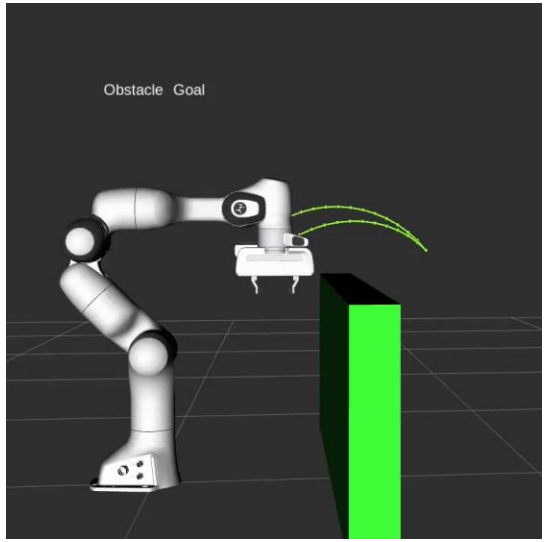
RRT: https://wiki.ros.org/rrt_exploration

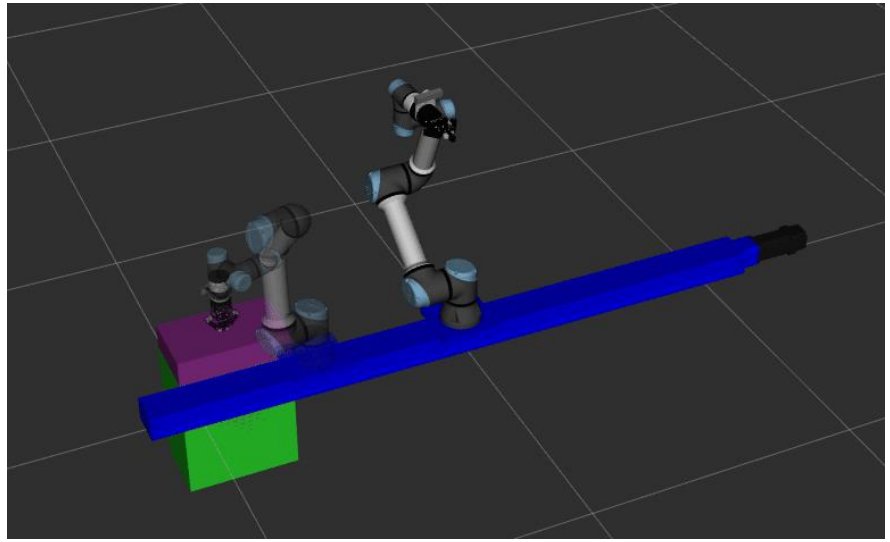
MoveIt https://moveit.ai/

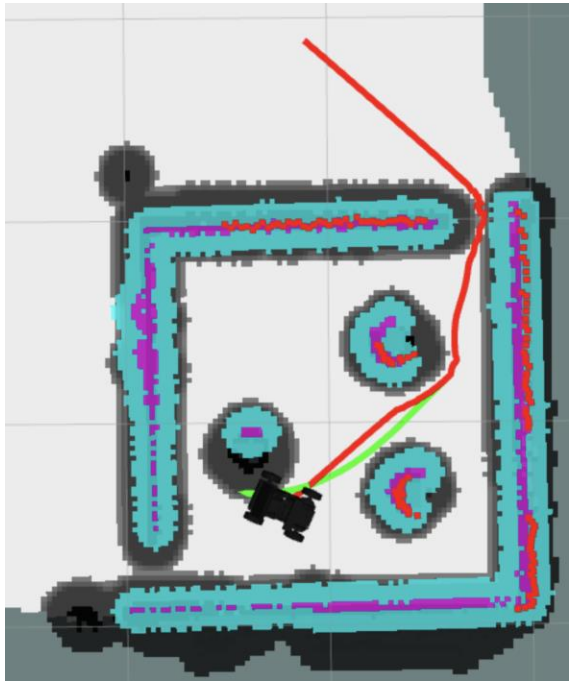
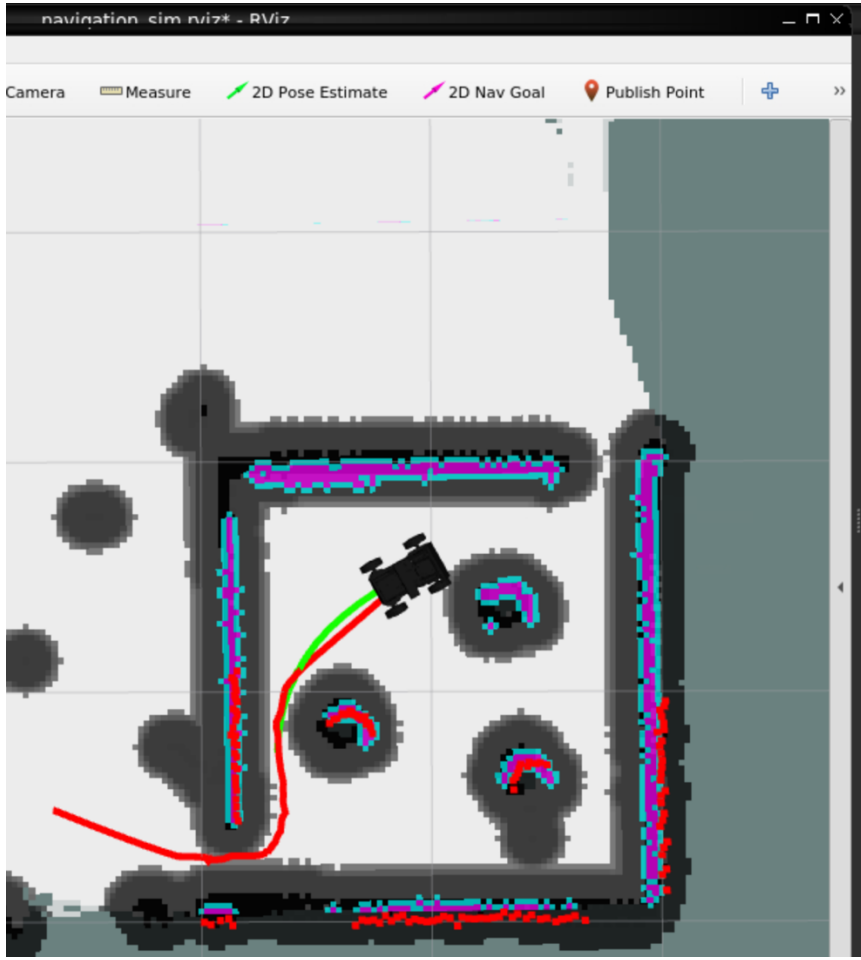
(particularly for manipulation)

Open Motion Planning Library in ROS MoveIt

- RRT*
- PRM*
- LazyPRM*
- BFMT
- FMT
- Lower Bound Tree RRT (LBTRRT)
- SPARS
- SPARS2
- Transition-based RRT (T-RRT)
- Adaptively Informed Tree (AIT*)
- Batched Informed Tree (BIT*)
- Advanced Batched Informed Tree (ABIT*)





ROS Can Help

Demo – Autonomous Navigation